Days 7–8 — Cost, Performance and Governance

Self-study edition, 30 Oct 2026 · Skills **4.1.1–4.1.4**, **4.2.1–4.2.6**, **3.3.1–3.3.4** · Exam: Mon 9 Nov, 1:30 PM IST

How to use this PDF offline
Read one part at a time. After each part, answer its check questions on paper. At the end, take the 12-question quiz (25 minutes, no notes) and send your answers when you're back online — the answers and explanations will be added then. Skim the glossary first if any term is unfamiliar.

Part 1 — Cutting costs (Task 4.1)

On-demand Bedrock bills per input token and per output token, so cost depends on how many tokens you send, how many you get back, and how many calls you make.

```
cost = (input tokens x input price) + (output tokens x output price)   per call
     x number of calls
Levers: (1) fewer/cheaper tokens  (2) cheaper model per call  (3) fewer calls (caching)
        (4) cheaper capacity (batch / Provisioned Throughput)
```

1.1 Token efficiency (4.1.1)

- **Measure first**: every Converse response includes usage (inputTokens, outputTokens). Log it per request and per feature as CloudWatch metrics to see which feature costs what. Estimate big prompts beforehand with token counting (supported models).
- **Context window optimisation**: send only the top relevant chunks, not whole documents; drop instructions that don't apply.
- **Context pruning** for long chats — you pay for the whole history on every call: sliding window (last N turns), summarise older turns, drop stale tool outputs.
- **Prompt compression**: same meaning, fewer tokens — remove repetition, fewer/shorter few-shot examples.
- **Response size controls**: output tokens cost more — sensible max tokens per feature, concise formats ('max 3 bullets'), JSON with only the needed fields.

```
Turn 30 WITHOUT pruning: [system] + [29 old turns ~12,000 tokens] + [new question]   <- paid every call
Turn 30 WITH pruning:    [system] + [summary of turns 1-25 ~300 tokens] + [last 4 turns] + [question]
```

Example: a support bot at 9,000 input tokens per turn drops to 2,500 with top-5 chunks + a history summary, and to 1,800 with a compressed system prompt — about 80% less input cost.

1.2 Cost-effective model selection (4.1.2)

Compare **cost per successful task**, not price per token — a cheap model that often fails and triggers retries or escalation can cost more.

```
Model A: $0.002/request, 95% success                          -> ~$0.0021 per success
Model B: $0.0008/request, 60% success (+ escalate failures to A) -> 0.0008 + 0.4 x 0.002 = $0.0016
```

- Tiered usage by complexity: routing and cascading (gap lesson).
- Balance cost against quality with evaluation: cheapest model that meets the quality bar.
- Efficient patterns: batch inference (~50% cheaper) for non-urgent work; caching.

1.3 Throughput and capacity (4.1.3)

- **Batching**: batch inference jobs; or group many small items in one prompt (classify 20 short reviews in one call returning a JSON list) so instructions are paid for once.

- **Capacity planning**: needed TPM ≈ peak requests per minute × average tokens per request (e.g. 300 × 2,000 = 600,000 TPM) — compare with quotas or plan Provisioned Throughput.
- **Utilisation monitoring**: Bedrock CloudWatch metrics — Invocations, InputTokenCount, OutputTokenCount, InvocationLatency, InvocationThrottles.
- **Auto-scaling** for self-hosted models on SageMaker AI: scale on load; keep a warm minimum (slow model loading).
- **Provisioned Throughput optimisation**: buy for the steady baseline only, send bursts to on-demand or cross-Region inference; commit only when usage is stable; monitor utilisation.

1.4 Caching — the cheapest call is the one you don't make (4.1.4)

```
request -> (1) edge cache (CloudFront)      identical public response?      -> return instantly
        -> (2) exact-match cache (hash)     same normalised prompt + params? -> stored answer
        -> (3) semantic cache (embeddings)  SIMILAR question answered?       -> stored answer
        -> (4) model call + PROMPT CACHING  reuse processed static prefix    -> cheaper, faster
```

- **Prompt caching** (Bedrock): mark a cache checkpoint after a long static prefix (system prompt, document, tool definitions); later calls reuse it — cached input tokens are discounted and responses start faster. The cache lasts minutes and refreshes on use. Put static content first, the user's question last — earlier changes break the cache. Example: the same 30-page contract sent with every question.
- **Exact-match cache** (deterministic request hashing / result fingerprinting): normalise the request, hash it, look it up in ElastiCache or DynamoDB; best with temperature 0.
- **Semantic cache**: embed the question, search previously answered questions (ElastiCache vector search or OpenSearch); above a similarity threshold, return the cached answer. Too low a threshold returns wrong answers to merely similar-looking questions.
- **Edge caching** with CloudFront for public, identical, non-personal responses.
- **Cautions**: never serve one user's personalised or PII answer to another (user/tenant in the key, or don't cache); set a TTL; invalidate on data changes (e.g. after a KB sync).

Part 1 check questions (answer on paper)

1. Why can a cheaper model end up costing more overall?
2. A legal assistant sends the same 50-page contract with every question. Which feature cuts cost, and where should the user's question go in the prompt?
3. What is the risk of a semantic cache with a low similarity threshold?

Part 2 — Making it fast (Task 4.2)

Where the time goes in one LLM call: a short wait before the first token (TTFT, grows with input size), then generation, which takes time **per output token**. That's why output length is the biggest latency lever.

```
|--- network ---|--- read prompt (TTFT) ---|--- generate token, token, token ... ---|
                  more input = longer        more OUTPUT tokens = proportionally longer
```

2.1 Responsive systems (4.2.1)

- **Pre-computation for predictable queries**: generate answers ahead of time (nightly) for the top questions, daily summaries or product descriptions, store them, serve instantly.
- **Latency-optimised models**: Bedrock offers a latency-optimised inference option for some models in some Regions (a performance setting on the call) for time-sensitive apps; otherwise choose smaller, faster models.
- **Parallel requests**: run independent sub-tasks concurrently (Step Functions Parallel or Map states, async SDK calls) — total time ≈ the slowest call, not the sum.
- **Response streaming**: improves perceived latency (time to first token).

- **Performance benchmarking**: measure TTFT, tokens per second, p50/p95/p99 latency under realistic load before choosing a model or configuration.

```
Sequential: [summary 4s] -> [risks 4s] -> [actions 4s] = 12 s
Parallel:   [summary 4s]
            [risks   4s]  -> merge -> ~4-5 s
            [actions 4s]
```

2.2 Retrieval performance (4.2.2)

- **Index optimisation**: ANN settings (HNSW, ef_search), shards and replicas, vector quantisation, per-domain indexes (gap lesson).
- **Query preprocessing**: rewrite/expand, apply metadata filters first so the vector search covers less data.
- **Hybrid search with custom scoring**: weight keyword vs vector results and boost by fields such as recency or document type (e.g. newer policies rank higher).

2.3 Throughput (4.2.3)

- **Token processing optimisation**: shorter prompts and outputs fit more requests within the same TPM quota.
- **Batch inference** for bulk work.
- **Concurrent invocation management**: cap concurrency so you stay within quotas — SQS-driven workers, Lambda reserved concurrency as a ceiling, token-bucket style rate limiting, retries with jitter to avoid synchronized retry storms.

2.4 Model-specific tuning (4.2.4) — recap

Parameters per use case (Day 5): low temperature for extraction/classification, higher for creative work; top-p / top-k to narrow choices; max tokens per feature. Use A/B tests to prove a change improves quality or latency.

2.5 Resource allocation (4.2.5)

- **Capacity planning for token processing**: profile each feature's input vs output tokens (summarisation is input-heavy; drafting is output-heavy) and plan quotas or model units from that.
- **Utilisation monitoring of prompt and completion patterns**: watch InputTokenCount vs OutputTokenCount trends to spot prompt bloat or runaway answer length.
- **Auto-scaling tuned for GenAI traffic** (self-hosted): scale on concurrency or invocations, not CPU; scheduled scaling before known peaks; warm minimum because large models load slowly.

2.6 System performance (4.2.6)

- **API call profiling**: break each call into TTFT vs generation time and trace across services with X-Ray to find the slow hop.
- **Vector query optimisation**: filter first, keep k (number of results) small, avoid returning huge payloads.
- **LLM latency reduction**: shorter outputs (biggest lever), smaller or distilled models, prompt caching (faster start for long static prefixes), streaming, latency-optimised inference, same-Region calls.
- **Efficient service communication**: avoid long chains of synchronous hops; use async/event-driven steps; reuse SDK clients and connections instead of creating them per request.

Part 2 check questions

1. Why does capping output length usually reduce latency more than shortening the input?
2. A report needs a summary, a risk list and an action list, each from a separate model call. How do you cut total time?

3. Your Lambda functions call Bedrock and burst to thousands of concurrent calls, causing throttling. Name two ways to manage concurrency.

Part 3 — Governance and compliance (Task 3.3)

Governance answers four questions an auditor or regulator will ask: **What does this model do and not do? Where did its data come from? Who approved it and who is accountable? How do we know it is still behaving?**

3.1 Compliance frameworks (3.3.1)

- **Programmatic model cards** with SageMaker AI: create and update model cards through the API/SDK inside your pipeline, so intended use, training data, evaluation results and limitations are recorded automatically for every version (not hand-written once and forgotten).
- **Automatic data lineage with AWS Glue**: Glue jobs and the Glue Data Catalog record which sources fed which datasets (and SageMaker lineage tracking links data, training jobs and models) — answers 'where did this come from?'
- **Metadata tagging for source attribution**: tag documents and chunks with source system, owner, date, classification — so every answer can be traced to its sources.
- **Decision logs in CloudWatch Logs**: for each important answer log the request, retrieved sources, model and guardrail versions, output and resulting decision — a complete record for audits.

3.2 Data source tracking (3.3.2)

- **Register sources in the Glue Data Catalog** (one inventory of datasets, schemas, owners).
- **Attribution in generated content**: citations and source IDs from metadata attached to answers.
- **CloudTrail** for audit logging of who accessed or changed data, models and guardrails.

```
Source systems -> Glue Data Catalog (registered, owner, schema) -> pipeline (lineage recorded)
  -> KB chunks tagged {source, owner, date} -> answer with citations -> decision log (CloudWatch Logs)
  -> every API action recorded by CloudTrail
```

3.3 Organisational governance (3.3.3)

A governance framework aligns AI work with company policy, regulation and responsible-AI principles:

- Roles and accountability: an AI governance board or review group; named owners per model and use case.
- Risk assessment per use case before build and before launch (impact, data sensitivity, failure modes).
- Approval workflow for models and prompts (e.g. Model Registry approval status; prompt version approval).
- Standards enforced technically: approved-model list via SCPs/IAM, mandatory guardrails, logging and encryption baselines checked by AWS Config rules.
- Evidence for audits: AWS Audit Manager can collect evidence continuously against frameworks (including a generative AI best-practices framework).
- Documentation and review cadence: model cards, periodic re-evaluation.

3.4 Continuous monitoring and advanced controls (3.3.4)

- **Automated misuse detection**: alarms on guardrail intervention rates, unusual usage per user or API key.
- **Drift detection**: track quality and grounding scores, response length and topics over time; for self-hosted models SageMaker Model Monitor detects data and model quality drift.
- **Bias drift monitoring**: fairness metrics tracked over time (SageMaker Clarify with Model Monitor for bias drift on hosted models).

- **Automated alerting and remediation**: CloudWatch alarm → EventBridge → Lambda or Systems Manager automation (roll back a model version, disable a feature flag, open a ticket).
- **Token-level redaction**: mask PII in prompts, responses and logs (Guardrails masking, Comprehend) so logs remain usable without exposing data.
- **Response logging**: model invocation logging, encrypted with KMS, retention via lifecycle rules.
- **AI output policy filters**: guardrails enforcing output policies.

```
metric/alarm (drift, misuse, bias gap) -> EventBridge rule -> Lambda / SSM Automation
     -> roll back model version | flip AppConfig flag off | notify + open ticket
```

Part 3 check questions

1. An auditor asks which source documents and which model version produced a specific answer last March. What must you have been logging?
2. Why create model cards programmatically in the pipeline instead of writing them by hand?
3. Grounding scores fall steadily over two weeks after a data change. Describe an automated detect-and-respond chain.

Decision rules — Days 7–8

1. Find costly features → log token usage per feature; long chats → context pruning; big inputs → relevant chunks + compression; output cost → max tokens + concise formats.
2. Compare models by cost per successful task; tiered usage via routing/cascading.
3. Many tiny calls → group items per prompt or batch inference; plan TPM = peak requests × tokens.
4. Provisioned Throughput for the steady baseline only; monitor utilisation.
5. Same long prefix each call → prompt caching (static first). Identical requests → exact-match cache. Similar questions → semantic cache with a careful threshold. Public identical pages → CloudFront. Never cache personal answers across users; TTL + invalidation.
6. Predictable queries → pre-compute. Time-critical → latency-optimised or smaller model + streaming. Independent sub-tasks → parallel calls.
7. Slow retrieval → filter first, tune ANN/index, hybrid scoring with boosts.
8. Throttling under bursts → cap concurrency (SQS workers, reserved concurrency), backoff with jitter.
9. Latency → shorten outputs first; profile TTFT vs generation; X-Ray across hops; reuse clients.
10. Model documentation → programmatic SageMaker model cards; lineage → Glue / Data Catalog; attribution → metadata tags + citations; audit → CloudTrail + decision logs in CloudWatch Logs.
11. Organisational governance → roles, risk assessment, approvals, enforced standards (SCP, Config), Audit Manager evidence.
12. Continuous governance → drift, bias-drift and misuse alarms → EventBridge → automated remediation; redact PII in logs.

Quiz — 12 questions (25 minutes, answers added when you send yours)

Question 1

A support assistant's monthly cost doubled. Conversations are long, and every call resends the full history. Users say answer quality is fine. What should the team do first?

A. Switch to a larger model so that fewer follow-up questions are needed

B. Purchase Provisioned Throughput sized to the current token volume

C. Raise the maximum output tokens so that each answer covers more follow-up questions

D. Keep the recent turns verbatim and replace older turns with a short running summary

Question 2

A legal team asks many questions about the same 60-page contract within a session. Each call includes the full contract followed by the question. What reduces cost and latency with the least change?

A. Move the question before the contract so that the model reads it first

B. Store the contract in a knowledge base and retrieve the whole contract on every call

C. Place the contract first with a cache checkpoint after it, and put the question last

D. Summarise the contract once and send only the summary with each question

Question 3

An internal help desk receives thousands of reworded versions of the same twenty questions each day. Answers aren't personalised and change only when policies are updated. What avoids most model calls safely? (Select TWO.)

A. Hash each question's exact text and return a stored answer only on identical text

B. Embed each question and return a stored answer when a previous question is sufficiently similar

C. Lower the similarity threshold as far as possible so that more questions match

D. Clear the stored answers whenever the policy knowledge base is re-synced

E. Cache answers at the edge with CloudFront using each user's session cookie as the key

Question 4

Model A costs $0.004 per request with 96% task success. Model B costs $0.0015 with 70% success; failures are re-run on Model A. Which statement is correct?

A. Model A is cheaper per successful task because it rarely fails

B. Model B with escalation costs about $0.0027 per request, which is cheaper than Model A

C. Both cost the same because failures cancel out the price difference

D. Model B is always cheaper because its price per request is lower

Question 5

A dashboard shows the same morning market summary to 50,000 employees. It currently calls the model for every page view. What should the team do?

A. Generate the summary once each morning, store it, and serve the stored copy

B. Use streaming so that each employee sees the summary appear faster

C. Switch to a latency-optimised model for every page view

D. Add a semantic cache keyed on each employee's ID

Question 6

A report needs three independent model outputs (summary, risks, actions), each taking about 5 seconds. Users wait 15 seconds. What should the team do?

A. Combine all three instructions into one longer prompt with a larger output limit

B. Switch the three calls to batch inference

C. Increase the Lambda memory of the function that makes the calls

D. Run the three calls concurrently, for example with a Step Functions Parallel state, then merge the results

Question 7

Thousands of Lambda invocations call Bedrock at once during a nightly job, causing throttling and failed records. The job has several hours to finish. Which approach fits best?

A. Increase Lambda reserved concurrency to the maximum so that the job finishes sooner

B. Retry every failure immediately without delay until it succeeds

C. Feed records through an SQS queue processed by a capped number of workers, with backoff and jitter on throttles

D. Purchase Provisioned Throughput for the full peak of the nightly burst

Question 8

A chat app's p95 latency is high. Traces show the time is mostly spent generating long answers, while users read only the first few paragraphs. What is the most effective fix?

A. Cap answer length with a lower max tokens and an instruction for concise answers

B. Add replicas to the vector index

C. Increase the number of retrieved chunks

D. Move the application to a Region farther from users but with more capacity

Question 9

Auditors require, for any answer from the last year, the sources used, the model and guardrail versions, and the decision taken. What should the team implement?

A. CloudTrail management events for the Bedrock account

B. A model card updated quarterly

C. Monthly LLM-as-a-judge evaluations

D. Structured decision logs in CloudWatch Logs per answer, with source IDs from document metadata

Question 10

A company retrains a model weekly and wants its documentation (intended use, data, evaluation results, limitations) always current for every version without manual effort. What should it do?

A. Write a model card by hand at the first release and link it from every later version

B. Create SageMaker model cards through the API as a step in the training pipeline

C. Store the training notebooks in S3 as the documentation

D. Enable CloudTrail data events on the training bucket

Question 11

A company must prove where every dataset used by its GenAI pipelines came from and how it was transformed. What should it use?

A. Amazon Macie discovery jobs on the source buckets

B. S3 Lifecycle rules on the raw data bucket

C. AWS Glue Data Catalog registration with lineage tracked by its Glue jobs

D. Bedrock model invocation logging

Question 12

After a data change, grounding scores drop steadily, and the company wants this detected and contained without waiting for humans. Which combination should it use? (Select THREE.)

A. Publish grounding scores as CloudWatch metrics with an alarm on a sustained drop

B. A quarterly manual review of a sample of answers

C. An EventBridge rule that reacts to the alarm

D. Raising the guardrail grounding threshold to its maximum permanently

E. A Lambda or Systems Manager automation that rolls back to the previous data or model version or disables the feature flag

F. Model invocation logging alone

Results — check questions and quiz (graded 30 Oct)

Part	Your answers	Feedback
1	Cost per success; prompt caching order; low semantic threshold	✓ ✓ ½ — a low threshold returns the SAME cached answer to questions that only look similar (wrong answers), not different answers.
2	Output length; parallel calls; concurrency	✓ ½ ✓ — parallel total time ≈ the SLOWEST call (not the shortest), plus merging.
3	Audit trail; programmatic model cards; drift response	½ ✓ ✓ — per-answer audits need DECISION LOGS (source IDs, model and guardrail versions, output, decision) in CloudWatch Logs; Glue lineage explains where datasets came from, not a single answer.

Quiz — 8 / 12 (67%)

Q	Skill	You	Correct	Why
1	4.1.1	D ✓	D	Prune/summarise old turns; quality is fine, history is the cost.
2	4.1.4	C ✓	C	Prompt caching: static contract first with checkpoint, question last.
3	4.1.4	A, D ✗	B, D	Questions are REWORDED → exact-text hashing rarely hits; semantic cache + invalidate on KB sync.
4	4.1.2	A ✗	B	A: 0.004/0.96 ≈ $0.0042 per success. B: 0.0015 + 0.30 × 0.004 = $0.0027 — cheaper despite failures.
5	4.2.1	A ✓	A	Pre-compute once, serve the stored copy.
6	4.2.1	D ✓	D	Parallel calls; time ≈ slowest call.
7	4.2.3	C ✓	C	SQS + capped workers + backoff with jitter; hours available.
8	4.2.6	C ✗	A	Time is spent generating long answers → cap output length (you stated this rule in Part 2!).
9	3.3.1	B ✗	D	Per-answer audit = structured decision logs with source IDs and versions; a model card is model-level documentation.
10	3.3.1	B ✓	B	Programmatic SageMaker model cards in the pipeline.
11	3.3.2	C ✓	C	Glue Data Catalog + lineage from Glue jobs.
12	3.3.4	A, C, E ✓	A, C, E	Metric + alarm → EventBridge → automated rollback/flag off.

Feedback
8/12 after **self-study only** — below the 75% target, as expected without a taught lesson. Two misses (Q8, Q9) contradict rules you stated correctly in the check questions → slow down and map each option to the symptom in the stem. Re-read: caching types (exact vs semantic), cost-per-success arithmetic, and decision logs vs model cards vs lineage.

Glossary — new terms

Term	Meaning
Token usage / token counting	Input/output token counts returned per call; estimating tokens before sending.
Context pruning / sliding window	Dropping or summarising old conversation turns to cut tokens.
Prompt compression	Same instructions in fewer tokens.
Cost per successful task	Total cost including failures, retries and escalations divided by successes.
Capacity planning	Estimating needed tokens/requests per minute from peak traffic.
Bedrock CloudWatch metrics	Invocations, InputTokenCount, OutputTokenCount, InvocationLatency, InvocationThrottles.
Prompt caching / cache checkpoint	Reusing a processed static prompt prefix; the marker after the cached part.
Exact-match cache / request hashing / fingerprint	Cache keyed by a hash of the normalised request.
Semantic cache	Cache matched by embedding similarity of questions.
TTL / cache invalidation	Time-to-live for cached items / removing them when data changes.
Pre-computation	Generating answers ahead of time for predictable requests.
Latency-optimised inference	Bedrock option for faster responses on supported models/Regions.
TTFT / tokens per second	Time to first token / generation speed.
p50 / p95 / p99	Latency percentiles (median, 95th, 99th).
Parallel / Map state	Step Functions states running branches or items concurrently.
Concurrency control / reserved concurrency	Capping simultaneous calls; Lambda's per-function concurrency setting.
Custom scoring / boosting	Adjusting search ranking by weights or fields such as recency.
Programmatic model card	Model card created/updated by code in a pipeline.
Data lineage	Record of where data came from and how it was transformed.
AWS Glue Data Catalog	Central inventory of datasets and schemas.
Decision log	Per-answer record of inputs, sources, versions, output and decision.
AWS Config / AWS Audit Manager	Configuration compliance rules / continuous audit evidence collection.
SageMaker Model Monitor	Detects data, quality and bias drift for hosted models.
Drift / bias drift	Gradual change in data or behaviour / in fairness metrics over time.
Automated remediation	Alarm → EventBridge → Lambda/SSM action without human delay.
Token-level redaction	Masking individual sensitive tokens (PII) in text and logs.